

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ



ETL ΣΕ ΡΟΕΣ ΔΕΔΟΜΕΝΩΝ ΧΡΗΣΙΜΟΠΟΙΩΝΤΑΣ ΤΟ ΑΡΑΧΕ FLINK
ETL ON DATA STREAMS USING APACHE FLINK

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ
του
ΑΘΑΝΑΣΙΟΥ ΜΠΟΥΤΑ

Εξεταστική επιτροπή
Καθηγητής Αντώνιος Δεληγιαννάκης (επιβλέπων)
Καθηγητής Μίνως Γαροφαλάκης
Επίκουρος Καθηγητής Γιατράκος Νίκος

Περίληψη

Abstract

Commented [AD1]: Πέραν του ότι λείπει, βάλε και μία αφιέρωση στους γονείς σου 😊

Περιεχόμενα

Κεφάλαιο 1 Εισαγωγή	7
Αντικείμενο διπλωματικής.....	7
Κεφάλαιο 2 Θεωρητικό υπόβαθρο.....	9
2.1 Ροές δεδομένων (data streams).....	9
2.2 Επεξεργασία ροών δεδομένων	9
2.3 Συστήματα ETL	9
2.4 Apache Kafka	10
2.5 Apache Flink	11
Κεφάλαιο 3 Λειτουργικές απαιτήσεις	13
3.1 Διαχείριση και επεξεργασία εισερχόμενων ροών δεδομένων.	13
3.2 Δημιουργία δεδομένων αισθητήρων σε πραγματικό χρόνο.....	13
3.3 Δυναμική υποβολή μετασχηματισμών	14
3.3 Εξοδος και αποθήκευση αποτελεσμάτων.....	16
Κεφάλαιο 4 Σχεδιασμός και υλοποίηση	17
4.1 Επισκόπηση συστήματος	17
4.2 Flink Job.....	17
4.2.1 CoFlatMap Operator	18
4.2.2 Εφαρμογή μετασχηματισμών	20
4.2.3 Παράθυρα	22
4.2.4 Αποθήκευση αποτελεσμάτων	23
4.3 Ροή Δεδομένων End-to-End.....	24
4.4 Βοηθητικές υπηρεσίες - Microservices	25
4.4.1 ETL API Service	26
4.4.2 File Producer Service	27
4.5 Docker Containerization	28
4.6 Τεχνολογικό Stack.....	29
Κεφάλαιο 5 Εξαγωγή πειραμάτων.....	30
5.1 Πειραματικά αποτελέσματα.....	30
5.1.1 Επίδραση του επιπέδου παραλληλισμού	31

5.1.2 Επίδραση του μεγέθους δεδομένων εισόδου	32
5.1.3 Επίδραση του αριθμού ενεργών configurations.....	33
5.1.4 Επίδραση των ενεργών configurations και της πολυπλοκότητάς τους	Error! Bookmark not defined. 34
Κεφάλαιο 6 Επίλογος.....	35
6.1 Συμπεράσματα.....	35
6.2 Μελλοντικές επεκτάσεις	35
Βιβλιογραφία.....	37

Κεφάλαιο 1

Εισαγωγή

Στις μέρες μας η πληροφορία και η αξιοποίηση των δεδομένων αποτελούν κρίσιμους παράγοντες για τη λήψη αποφάσεων σε οργανισμούς και επιχειρήσεις. Με την ραγδαία άνοδο του όγκου **τις** πληροφορίας, τα δεδομένα προς επεξεργασία συνήθως εμφανίζονται ως μια συνεχής ροή γεγονότων. Συναλλαγές με πιστωτικές κάρτες, μετρήσεις αισθητήρων, αλληλεπιδράσεις χρηστών σε ιστοσελίδες και εφαρμογές. Η επεξεργασία τέτοιων ροών παρουσιάζει σημαντικές προκλήσεις, καθώς τα δεδομένα είναι θεωρητικά άπειρα, εισέρχονται συνεχώς στο σύστημα και πρέπει να επεξεργαστούν σε πραγματικό ή σχεδόν πραγματικό χρόνο. Για την αντιμετώπιση αυτών των προκλήσεων έχουν αναπτυχθεί πολλές πλατφόρμες καταμεμημένης επεξεργασίας. Η παρούσα εργασία βασίζεται στο Apache Flink [1].

Συχνά, είναι επιτακτική η ανάγκη της επεξεργασίας και εξαγωγής χρήσιμων αποτελεσμάτων από μεγάλο όγκο δεδομένων σε πραγματικό χρόνο, με απώτερο στόχο τη λήψη σημαντικών αποφάσεων. Ειδικότερα, στον τομέα της παρακολούθησης δικτύων αισθητήρων, ζωτικής σημασίας είναι η έγκαιρη ανίχνευση ανωμαλιών στις μετρήσεις, η οποία μπορεί να αποτρέψει βλάβες εξοπλισμού, να εξασφαλίσει την ασφάλεια και να βελτιστοποιήσει την ενεργειακή κατανάλωση. Στην πραγματικότητα, ο εκάστοτε διαχειριστής συστήματος ενδιαφέρεται για την εφαρμογή συγκεκριμένων κανόνων φιλτραρίσματος και συνάθροισης στα δεδομένα των αισθητήρων σε πραγματικό χρόνο, οι οποίοι μπορεί να μεταβάλλονται δυναμικά ανάλογα με τις εκάστοτε ανάγκες.

Αντικείμενο διπλωματικής

Το αντικείμενο της παρούσας διπλωματικής εργασίας επικεντρώνεται στην ανάπτυξη ενός ολοκληρωμένου συστήματος ETL (Extract, Transform, Load) για την επεξεργασία συνεχών ροών δεδομένων που προέρχονται από μετρήσεις αισθητήρων.

Σέ πρώτο επίπεδο έχει αναπτυχθεί η ροή δεδομένων που προσομοιώνει ένα πραγματικό δίκτυο αισθητήρων διαφόρων τύπων (θερμοκρασίας, υγρασίας, πίεσης, ποιότητας αέρα, φωτεινότητας, θορύβου), τα οποία αποστέλλονται συνεχώς σε πραγματικό χρόνο στο σύστημα μας προς επεξεργασία.

Αυτό που επιδιώκεται είναι η δυνατότητα δυναμικής υποβολής κανόνων μετασχηματισμού (configurations), οι οποίοι εφαρμόζονται στη ροή δεδομένων χωρίς να απαιτείται επανεκκίνηση της εφαρμογής. Στην ουσία, οι χρήστες πρέπει να μπορούν να ορίσουν φίλτρα, κανονικοποιήσεις και συναρτήσεις συνάθροισης σε πραγματικό χρόνο, επιτυγχάνοντας ουσιαστικά την υποβολή ερωτήσεων (queries) στην ροή των δεδομένων.

Προκειμένου να υλοποιηθούν τα παραπάνω, αξιοποιείται το Apache Flink ως μηχανή επεξεργασίας ροών και το Apache Kafka για τη διαμεσολάβηση μηνυμάτων μεταξύ των

υπηρεσιών. Επιπλέον, εξετάζεται η απόδοση του συστήματος μεταβάλλοντας διάφορες παραμέτρους, όπως το επίπεδο παραλληλισμού, το μέγεθος των δεδομένων εισόδου και τον αριθμό των ενεργών configurations, ώστε να αξιολογηθεί η επίδρασή τους στον χρόνο επεξεργασίας.

Κεφάλαιο 2

Θεωρητικό υπόβαθρο

2.1 Ροές δεδομένων (data streams)

Ως ροές χαρακτηρίζονται οι συνεχείς ακολουθίες γεγονότων, ταξινομημένες χρονικά. Όταν αναφερόμαστε σε ροές δεδομένων εννοούμε τη συνεχή μεταφορά δεδομένων από ένα ή περισσότερα συστήματα, συνήθως με υψηλό ρυθμό, με σκοπό την επεξεργασία και την άντληση χρήσιμων πληροφοριών.

Η επεξεργασία ροών δεδομένων (stream processing) έχει εξελιχθεί σημαντικά τα τελευταία χρόνια, με την ανάπτυξη πολλών εξειδικευμένων λύσεων και πλατφορμών επεξεργασίας δεδομένων, που επιτρέπουν την επεξεργασία σε πραγματικό ή σχεδόν πραγματικό χρόνο, την ομαδική επεξεργασία (batch processing), αλλά και υβριδικές τεχνικές που συνδυάζουν και τα δύο μοντέλα. Ένα τέτοιο εργαλείο είναι και το Apache Flink το οποίο έχει χρησιμοποιηθεί στην παρούσα εργασία.

Παράλληλα, έχουν αναπτυχθεί διάφορες τεχνικές και ολοκληρωμένα συστήματα για την εξαγωγή, επεξεργασία και φόρτωση δεδομένων (Extract, Transform, Load – ETL).

Οι πληροφορίες και τα αποτελέσματα που προκύπτουν από την επεξεργασία ροών δεδομένων είναι καταλυτικές για οργανισμούς και επιχειρήσεις, καθώς επιτρέπουν σε αναλυτές ή τρίτα συστήματα να ενημερώνονται και να λαμβάνουν σημαντικές αποφάσεις σε πραγματικό ή σχεδόν πραγματικό χρόνο. Ενδεικτικά παραδείγματα αποτελούν συστήματα αισθητήρων ή χρηματοοικονομικές εφαρμογές.

2.2 Επεξεργασία ροών δεδομένων

Στην επεξεργασία ροών τα δεδομένα είναι θεωρητικά άπειρα και εισέρχονται συνεχώς στο σύστημα. Η επεξεργασία πραγματοποιείται πάνω σε γεγονότα ή μικρές ομάδες γεγονότων καθώς καταφθάνουν. Ως γεγονότα ορίζονται εγγραφές δεδομένων που ρέουν/εισέρχονται στο σύστημα. Οι ροές (Streams) είναι συνεχείς ακολουθίες γεγονότων, ενώ οι μετασχηματισμοί (Transformations) είναι πράξεις που εφαρμόζονται στα γεγονότα, όπως `map`, `filter` ή `flatMap`. Τέλος, η κατάσταση (state) θα μπορούσε να περιγραφεί ως η μνήμη, η οποία αφορά ουσιαστικά πληροφορίες που διατηρούνται μεταξύ των γεγονότων.

Commented [AD2]: Αυτά πρέπει να τα εξηγήσεις

2.3 Συστήματα ETL

Στην επιστήμη της πληροφορικής το Extract, Transform, Load (ETL), είναι μια υπολογιστική διαδικασία τριών φάσεων όπου τα δεδομένα αρχικά εισέρχονται από μία ή περισσότερες πηγές

ως είσοδος στο σύστημα, μετασχηματίζονται και τέλος φορτώνονται σε μία ή περισσότερες εξόδους. [2]

Στα παραδοσιακά συστήματα ETL η επεξεργασία γίνεται σε μεγάλες ομάδες δεδομένων (batches), με τα δεδομένα να συλλέγονται και να επεξεργάζονται ανά χρονικά διαστήματα που επιλέγει ο χρήστης. Πιο σύγχρονα εργαλεία, όπως είναι και το Apache Flink, επιτρέπουν την επεξεργασία σε πραγματικό χρόνο.

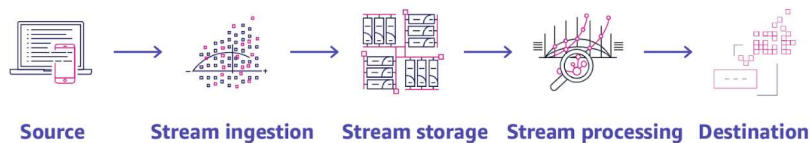
Τα streaming ETL συστήματα (Εικόνα 1) επιτρέπουν την επεξεργασία και φόρτωση των δεδομένων σε πραγματικό χρόνο. Η αρχιτεκτονική τους αποτελείται από τρία στοιχεία: πηγή δεδομένων, μηχανή streaming ETL και προορισμό.

- **Πηγή:** Είναι η προέλευση της ροής δεδομένων (π.χ. αισθητήρες, συσκευές IoT, κοινωνικά δίκτυα).
- **Μηχανή Streaming ETL:** Διαβάζει συνεχώς δεδομένα σε πραγματικό χρόνο, τα καθαρίζει, τα φιλτράρει και τα μετασχηματίζει καθώς φτάνουν.

Περιλαμβάνει:

- **Stream Ingestion:** π.χ. AWS IoT, Kinesis Agent, AWS SDK
- **Stream Storage:** π.χ. Amazon MSK, Kinesis Data Streams, Apache Kafka
- **Stream Processing:** π.χ. AWS Lambda για επεξεργασία γεγονότων

Μετά την επεξεργασία, τα δεδομένα φορτώνονται στον προορισμό για περαιτέρω ανάλυση (αποθήκη δεδομένων ή εφαρμογή βασισμένη σε γεγονότα). [3]



Εικόνα 1. Αρχιτεκτονική Streaming ETL

2.4 Apache Kafka

Το Apache Kafka είναι μια πλατφόρμα καταμεμημένης ροής γεγονότων (event streaming) ανοιχτού κώδικα, που χρησιμοποιείται ευρέως για την κατασκευή αγωγών δεδομένων υψηλής απόδοσης, αναλυτική επεξεργασία ροών δεδομένων, ενοποίηση δεδομένων [4].

Το Kafka συνδυάζει τρεις βασικές δυνατότητες:

- Δημοσίευση (write) και εγγραφή (read) σε ροές γεγονότων (events), συμπεριλαμβανομένης της συνεχούς εισαγωγής και εξαγωγής δεδομένων από άλλα συστήματα.

Commented [AD3]: Ποιος;

Commented [AD4]: Αν έχεις αντιγράψει την εικόνα από κάπου, πρέπει να βάλεις (Εικόνα από [http:// ...](http://...))

Επίσης, στο κείμενο ΠΡΕΠΕΙ να αναφέρεσαι και να περιγράφεις όλες τις εικόνες και τους πίνακες)

Formatted: English (United States)

- Αποθήκευση ροής γεγονότων με ανθεκτικό και αξιόπιστο τρόπο για όσο χρονικό διάστημα θέλει ο χρήστης.
- Επεξεργασία ροών γεγονότων τη στιγμή που συμβαίνουν ή και αναδρομικά.

Όλη αυτή η λειτουργικότητα παρέχεται με κατανεμημένο τρόπο, με υψηλή κλιμάκωση, ελαστικότητα, ανθεκτικότητα σε σφάλματα και ασφάλεια. Το Kafka μπορεί να αναπτυχθεί σε φυσικό εξοπλισμό (bare-metal), σε εικονικές μηχανές και σε containers, τόσο σε τοπικές υποδομές (on-premises) όσο και στο cloud [4].

Βασικά στοιχεία της αρχιτεκτονικής του είναι τα ακόλουθα:

- **Topics:** Λογικές δομές στις οποίες αποθηκεύονται οι εγγραφές.
- **Partitions:** Κάθε topic χωρίζεται σε ένα ή περισσότερα partitions ώστε να υπάρχει παραλληλισμός. Κάθε partition είναι ταξινομημένο.
- **Brokers:** Οι κόμβοι του Kafka cluster που αποθηκεύουν τα δεδομένα και εξυπηρετούν τους producers και consumers.
- **Producers:** Εφαρμογές που δημοσιεύουν μηνύματα σε topics.
- **Consumers:** Εφαρμογές που εγγράφονται σε topics και επεξεργάζονται τις εγγραφές.
- **Consumer Groups:** Επιτρέπουν την κατανομή της επεξεργασίας μεταξύ πολλαπλών καταναλωτών.
- **Zookeeper server:** Διαχειρίζεται τον Kafka cluster (σε νεότερες εκδόσεις δεν απαιτείται).

Το Kafka περιλαμβάνει πέντε βασικά APIs:

- **Producer API:** Επιτρέπει στις εφαρμογές να στέλνουν ροές δεδομένων (streams) σε topics μέσα στο Kafka cluster.
- **Consumer API:** Επιτρέπει στις εφαρμογές να διαβάζουν ροές δεδομένων από topics στον Kafka cluster.
- **Streams API:** Επιτρέπει τον μετασχηματισμό ροών δεδομένων από topics εισόδου σε topics εξόδου.
- **Connect API:** Επιτρέπει την υλοποίηση συνδέσεων (connectors) που είτε αντλούν συνεχώς δεδομένα από κάποιο σύστημα ή εφαρμογή προς το Kafka είτε προωθούν δεδομένα από το Kafka προς κάποιο σύστημα ή εφαρμογή-προορισμό.
- **Admin API:** Επιτρέπει τη διαχείριση και τον έλεγχο topics, brokers και άλλων αντικειμένων του Kafka [5].

2.5 Apache Flink

Το Apache Flink είναι μια πλατφόρμα και κατανεμημένη μηχανή επεξεργασίας δεδομένων, σχεδιασμένη για την εκτέλεση υπολογισμών με κατάσταση (stateful) τόσο σε συνεχείς όσο και σε πεπερασμένες ροές δεδομένων. Έχει αναπτυχθεί με στόχο να λειτουργεί αποδοτικά σε όλα τα συνήθη περιβάλλοντα συστάδων (clusters) και να εκτελεί υπολογισμούς με ιδιαίτερα υψηλή απόδοση, αξιοποιώντας κυρίως τη μνήμη (in-memory processing) [6].

Το Apache Flink διακρίνεται για την ευελιξία και την αποδοτικότητά του στην επεξεργασία ροών δεδομένων κάθε είδους. Η εκτέλεση μιας εφαρμογής στο Flink βασίζεται σε έναν κατευθυνόμενο άκυκλο γράφο (Directed Acyclic Graph – DAG), όπου κάθε κόμβος αντιστοιχεί σε έναν τελεστή (operator) επεξεργασίας και κάθε ακμή σε μία ροή δεδομένων. Η κατανομή των δεδομένων μεταξύ των τελεστών πραγματοποιείται μέσω μηχανισμών όπως οι keyBy, broadcast και rebalance, επιτρέποντας την υλοποίηση διαφορετικών στρατηγικών διαμερισμού και παραλληλισμού των δεδομένων.

Το περιβάλλον εκτέλεσης (runtime) της Flink αποτελείται από δύο βασικούς τύπους διεργασιών: τον JobManager και έναν ή περισσότερους TaskManagers. Ο JobManager είναι υπεύθυνος για τον προγραμματισμό και τον συνολικό συντονισμό της εκτέλεσης μιας εφαρμογής. Διαχειρίζεται το γράφημα εκτέλεσης του προγράμματος (JobGraph), παρακολουθεί την πρόοδο της επεξεργασίας, εκκινεί ή επανεκκινεί επιμέρους εργασίες (tasks) σε περίπτωση αποτυχίας και διασφαλίζει τη συνέπεια της κατάστασης μέσω του μηχανισμού checkpoints.

Οι TaskManagers (γνωστοί και ως workers) είναι υπεύθυνοι για την πραγματική εκτέλεση των εργασιών ενός dataflow, καθώς και για την προσωρινή αποθήκευση και την ανταλλαγή των ροών δεδομένων μεταξύ των επιμέρους σταδίων επεξεργασίας. Σε κάθε σύστημα Flink πρέπει να υπάρχει τουλάχιστον ένας TaskManager. Η μικρότερη μονάδα χρονοπρογραμματισμού πόρων σε έναν TaskManager είναι το λεγόμενο task slot, ενώ ο συνολικός αριθμός των διαθέσιμων slots καθορίζει τον μέγιστο αριθμό εργασιών που μπορούν να εκτελούνται ταυτόχρονα στο σύστημα (παραλληλισμός).

Κεφάλαιο 3

Λειτουργικές απαιτήσεις

Commented [AD5]: Εδώ θέλει λίγη δουλειά το κεφάλαιο αυτό

Στο παρόν κεφάλαιο περιγράφονται οι λειτουργικές απαιτήσεις του συστήματος Streaming ETL που αναπτύχθηκε στο πλαίσιο της παρούσας διπλωματικής εργασίας. Οι απαιτήσεις αυτές προέκυψαν ως αποτέλεσμα της ανάγκης, για τη δημιουργία ενός ρεαλιστικού προβλήματος, επεξεργασίας ροών δεδομένων σε πραγματικό χρόνο.

Στόχος, λοιπόν, του συστήματος που αναπτύχθηκε είναι η δημιουργία, συλλογή, επεξεργασία και αποθήκευση των αποτελεσμάτων που προέρχονται από συνεχή ροή δεδομένων προερχόμενες από ένα εικονικό δίκτυο αισθητήρων, με τρόπο αξιόπιστο, αποδοτικό και επεκτάσιμο. Στις επόμενες ενότητες παρουσιάζονται αναλυτικά οι βασικές λειτουργικές απαιτήσεις που καλείται να ικανοποιήσει το σύστημα.

3.1 Διαχείριση και επεξεργασία εισερχόμενων ροών δεδομένων.

Βασική λειτουργία, λοιπόν, που χρειάστηκε να αναπτυχθεί, είναι μία Flink εργασία, η οποία καταναλώνει μέσω Kafka δύο ροές. Πρόκειται για τη ροή δεδομένων αισθητήρων και τη ροή των ερωτήσεων που υποβάλει ο χρήστης.

Commented [AD6]: Αυτή η 1η παράγραφος που μιλάει για το Flink και τις 2 ροές είναι κομμάτι του Κεφαλαίου 4 κανονικά.
Η 2η παράγραφος αναφέρεται σε απαιτήσεις, οπότε είναι οκ εδώ.

Στόχος είναι κάθε νέο εισερχόμενο γεγονός της ροής να συνδυάζεται με κάθε αρχείο απαιτήσεων που έχει ήδη υποβληθεί από τον χρήστη, ενώ παράλληλα να επιτρέπει και την υποβολή νέων αρχείων χωρίς να χρειάζεται επανεκκίνηση η διαδικασία. Για κάθε ένα αρχείο χρήστη και για κάθε νέο γεγονός επιτρέπεται η εφαρμογή ενός ή περισσότερων μετασχηματισμών (φιλτράρισμα τιμών, εύρεση μέσης τιμής, κλπ.), ανάλογα με τις απαιτήσεις του χρήστη.

3.2 Δημιουργία δεδομένων αισθητήρων σε πραγματικό χρόνο

Μια από τις λειτουργίες του συστήματος αποτελεί η δημιουργία της ροής δεδομένων από αισθητήρες, και η προώθηση της στο Flink για περαιτέρω επεξεργασία σε πραγματικό χρόνο.

Για την εξυπηρέτηση του παραπάνω σεναρίου έχει δημιουργηθεί μια υπηρεσία (Service), που δημιουργεί την απαιτούμενη ροή δεδομένων. Τα δεδομένα αφορούν συγκεκριμένους τύπους αισθητήρων και τοποθεσίες όπου αυτοί είναι τοποθετημένοι. Επίσης, οι τιμές που παράγονται είναι κοντά σε αυτές του πραγματικού κόσμου (πχ, θερμοκρασία από -30°C , $+70^{\circ}\text{C}$).

Κάθε εγγραφή δεδομένων αισθητήρα περιλαμβάνει τα ακόλουθα πεδία:

- **Αισθητήρα (sensor)** με τις ακόλουθες τιμές:
 - Αισθητήρας θερμοκρασίας
 - Αισθητήρας υγρασίας
 - Αισθητήρας πίεσης

- ο Αισθητήρας ποιότητας αέρα
 - ο Αισθητήρας φωτεινότητας
 - ο Αισθητήρας θορύβου
- **Τοποθεσία προέλευσης (location)** με τις ακόλουθες τιμές:
 - ο room-a
 - ο room-b
 - ο room-c
 - ο room-d
 - ο room -e
 - ο room -f
 - ο room -g
 - ο room -h
- **Ένδειξη ποιότητας δεδομένων (data_quality)** με τις ακόλουθες τιμές:
 - ο Excellent
 - ο GoodFair
 - ο Poor
- **Τιμή μέτρησης (measurement)**
- **Μονάδα μέτρησης (measurement_unit)**
- **Ημερομηνία μέτρησης (datetime)**

3.3 Δυναμική υποβολή μετασχηματισμών

Το σύστημα μας θέλουμε να επιτρέπει στους χρήστες τη δυναμική υποβολή κανόνων μετασχηματισμού (configurations), μέσω μιας διεπαφής REST API σε μορφή JSON, χωρίς να απαιτείται επανεκκίνηση της εφαρμογής ή διακοπή της ροής επεξεργασίας.

Κάθε configuration περιλαμβάνει ένα jobId που αποτελεί μοναδικό αναγνωριστικό της εργασίας και μια λίστα μετασχηματισμών προς εφαρμογή.

Πεδία Configuration

Πεδίο	Τύπος	Περιγραφή
jobId	String	Μοναδικό αναγνωριστικό της εργασίας
Transformations	Array	Λίστα μετασχηματισμών προς εφαρμογή

Πίνακας 1: Πεδία configuration

Commented [AD7]: Αυτό είναι υλοποίηση και πάει στο Κεφάλαιο 4. Δεν είναι απαιτήσεις από το σύστημα.

Επίσης, έτσι που το γράφεις φαίνεται σαν να λειτουργεί η εφαρμογή σου ΜΟΝΟ για το συγκεκριμένο input.

Commented [AD8]: Ειλικρινά, αυτό θα έπρεπε να είναι τουλάχιστον 3-4 σελίδες πιο μεγάλο, να εξηγεί όλους τους μετασχηματισμούς, γιατί είναι χρήσιμοι, και να έχει παραδείγματα από configurations.

Formatted: English (United States)

Commented [AD9]: Αυτό πάλι είναι υλοποίηση

Commented [AD9R2]: Επίσης, παράδειγμα έχεις στο Κεφάλαιο 4 από configuration. Πρέπει να φροντίσεις να βάζεις το κείμενο στο σωστό μέρος.

Δομή Transformations

Κάθε μετασχηματισμός στη λίστα transformations περιλαμβάνει τα παρακάτω.

Πεδίο	Τύπος	Υποχρεωτικό	Περιγραφή
type	String	Ναι	Τύπος μετασχηματισμού
params	Object	Ναι	Παράμετροι του μετασχηματισμού
keyBy	String	Όχι	Πεδίο ομαδοποίησης (για aggregations)

Πίνακας 2: Δομή μετασχηματισμού

Υποστηριζόμενοι τύποι μετασχηματισμών

Element Transformations (εφαρμόζονται σε κάθε εγγραφή)

Τύπος	Περιγραφή	Παράμετροι
filter_greater	Διατηρεί εγγραφές όπου field > threshold	sensor, field, threshold
filter_less	Διατηρεί εγγραφές όπου field < threshold	sensor, field, threshold
Normalize	Κανονικοποίηση: (value - min) / (max - min)	field, min, max
to_lowercase	Μετατροπή string πεδίου σε πεζά	field
to_uppercase	Μετατροπή string πεδίου σε κεφαλαία	field
trim_whitespace	Αφαίρεση whitespace από αρχή/τέλος	field

Πίνακας 3: Element μετασχηματισμοί

Aggregation Transformations (εφαρμόζονται σε παράθυρο χρόνου)

Τύπος	Περιγραφή	Παράμετροι
sum	Άθροισμα τιμών	field, sensor (προαιρετικό)
max	Μέγιστη τιμή	field, sensor (προαιρετικό)
min	Ελάχιστη τιμή	field, sensor (προαιρετικό)
avg	Μέσος όρος	field, sensor (προαιρετικό)

Πίνακας 4: Aggregation μετασχηματισμοί

3.3 Έξοδος και αποθήκευση αποτελεσμάτων

Μετά την εφαρμογή των μετασχηματισμών, το σύστημα παράγει αποτελέσματα, τα οποία αποθηκεύονται σε μια βάση δεδομένων MongoDB.

Κάθε εγγραφή εξόδου περιλαμβάνει τα ακόλουθα πεδία αποτελέσματος (EtlResult):

Πεδίο	Τύπος	Περιγραφή
jobId	String	Αναγνωριστικό της εργασίας που παρήγαγε το αποτέλεσμα
transformations	Array	Οι μετασχηματισμοί που εφαρμόστηκαν
groupingField	String	Το πεδίο ομαδοποίησης (π.χ. "location", "sensor")
groupingKey	String	Η τιμή ομαδοποίησης (π.χ. "Kitchen")
aggregationType	String	Ο τύπος aggregation (sum, max, min, avg) - αν εφαρμόστηκε
field	String	Το πεδίο στο οποίο εφαρμόστηκε ο υπολογισμός
result	Object/Number	Το αποτέλεσμα της επεξεργασίας
processedAt	Timestamp	Χρονοσήμανση ολοκλήρωσης επεξεργασίας
sensorType	String	Ο τύπος του αισθητήρα
measurementUnit	String	Η μονάδα μέτρησης
location	String	Η τοποθεσία προέλευσης

Πίνακας 34: Πεδία εξόδου

Τύποι Αποτελεσμάτων

Ανάλογα με τους μετασχηματισμούς που εφαρμόζονται, το σύστημα παράγει δύο κατηγορίες αποτελεσμάτων:

- **Αποτελέσματα Element Transformations:** Όταν εφαρμόζονται μόνο element transformations (φίλτρα, μετατροπές κειμένου, κανονικοποίηση), το πεδίο result περιέχει την επεξεργασμένη εγγραφή του αισθητήρα.
- **Αποτελέσματα Aggregation Transformations:** Όταν εφαρμόζονται aggregations, το πεδίο result περιέχει την τιμή της συνάθροισης που υπολογίστηκε εντός του χρόνου του παραθύρου.

Commented [AD10]: Αυτό είναι απαίτηση. Όλες οι απαιτήσεις θα έπρεπε να είναι μαζεμένες σε κάποια υποενότητα.

Commented [AD11]: Γιατί χρειάζονται όλα αυτά τα πεδία; Αν δεν περιγράφεις τους πίνακες, αυτό δεν είναι καλό κείμενο.

Commented [AD11R2]: Θάβεις την προσπάθειά σου.

Η εξαγωγή του τύπου του αισθητήρα από τη ροή με τα configuration files αντίθετα ήταν λίγο πιο περίπλοκη. Δεδομένου ότι η παράμετρος 'sensor' αν και υποχρεωτική δεν περιέχεται απαραίτητα σε κάθε μετασχηματισμό, δημιουργήθηκε μια διαδικασία (ConfigKeyExtractor), που εξάγει για όλα τα configurations τον τύπο αισθητήρα ως κλειδί, από το πρώτο πεδίο/μετασχηματισμό που θα βρει να περιέχει την παράμετρο 'sensor'.

Για παράδειγμα, ένα config με {lower_case (location) και filter_greater (sensor:temperature)} θα πάρει key "temperature" από το δεύτερο transformation.

Με αυτόν τον τρόπο επιτυγχάνουμε ότι τα configs και events με τον ίδιο αισθητήρα δρομολογούνται στο ίδιο subtask του Flink, ώστε να γίνουν connect στο επόμενο στάδιο με τη χρήση του CoFlatMap operator, όπως αναλύεται εκτενώς στη συνέχεια.

4.2.1 CoFlatMap Operator

Ο operator αυτός δέχεται δύο εισόδους και διατηρεί μια εσωτερική κατάσταση (state). Όταν φτάνει ένα configuration, αποθηκεύεται στο state. Όταν φτάνει ένα sensor event, ανακτώνται τα configurations από το state και εφαρμόζονται οι αντίστοιχοι μετασχηματισμοί. Αυτή η αρχιτεκτονική επιτρέπει την διαρκή και ανεξάρτητη υποβολή αρχείων απαιτήσεων από τον χρήστη, χωρίς επανεκκίνηση του job. Τα τελικά αποτελέσματα της επεξεργασίας αποθηκεύονται στην βάση MongoDB.

Commented [AD14]: Εδώ ο τίτλος θα έπρεπε να είναι η λειτουργία που γίνεται σε αυτόν τον operator. Δεν είναι κεφάλαιο για το Flink αυτό.

Η CoFlatMapFunction αποτελεί βασικό μηχανισμό του Flink για τη συνένωση δύο ροών δεδομένων (connected streams), όπου κάθε ροή επεξεργάζεται από ξεχωριστή μέθοδο αλλά μοιράζονται κοινή κατάσταση [7].

Ο CoFlatMapProcessor που υλοποιήσαμε χρησιμοποιεί την RichCoFlatMapFunction. Η RichCoFlatMapFunction είναι η εμπλουτισμένη έκδοση της CoFlatMapFunction που παρέχει πρόσβαση σε managed state και lifecycle methods μέσω του RuntimeContext [8].

Στην περίπτωσή μας η χρήση της Rich έκδοσης παρέχει:

- Managed State: MapState για αποθήκευση configurations ανά jobId
- Lifecycle method open(): Αρχικοποίηση του state πριν την επεξεργασία δεδομένων

Η χρήση keyed state εξασφαλίζει ότι τα configurations και events με το ίδιο key επεξεργάζονται από τον ίδιο parallel task. Και οι δύο ροές χρησιμοποιούν το ίδιο κλειδί κατανομής: τον τύπο αισθητήρα. Τα events κατανέμονται με βάση το πεδίο sensor, ενώ τα configurations με βάση την παράμετρο sensor των μετασχηματισμών τους. Αυτή η στρατηγική keying εξασφαλίζει ότι ένα configuration που αφορά π.χ. θερμοκρασία θα βρίσκεται στο ίδιο parallel task με τα events θερμοκρασίας, επιτρέποντας την άμεση πρόσβαση στο state χωρίς ανάγκη επικοινωνίας μεταξύ tasks.

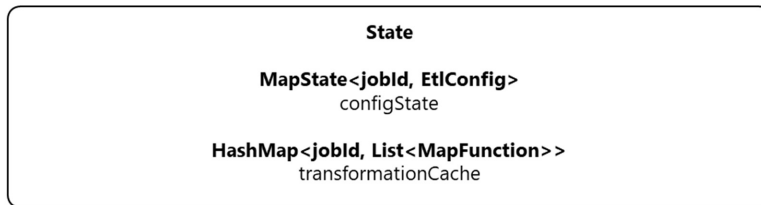
Η διαχείριση των δύο ροών γίνεται από τις παρακάτω map μεθόδους (Διάγραμμα 2).

- flatMap1 (Config)

Commented [AD15]: Θέλει να γραφτεί ώστε να αναφέρεται στη λειτουργία που υλοποιεί.

- Το αρχείο απαιτήσεων αποθηκεύεται στο `MapState<String, EtlConfig>` με κλειδί το `jobId`.
 - Δημιουργούνται οι transformations `MapFunction` και αποθηκεύονται με caching και κλειδί το `jobId`.
 - Εκπέμπει ένα `EtlResult` ως `config status confirmation`.
- **flatMap2 (Sensor)**
 - Για κάθε αρχείο απαιτήσεων στο state διαχωρίζει τα transformations σε element και aggregation.
 - Εφαρμόζει τα cached element transformation functions σειριακά. Αν κάποιο filter επιστρέψει null το event κόβεται και προχωράει στο επόμενο config.
 - Αν το event περάσει τα φίλτρα και υπάρχουν aggregations τότε εκπέμπεται `EtlResult` για κάθε matching aggregation (windowed path).
 - Αν το event περάσει τα φίλτρα και δεν υπάρχουν aggregations τότε εκπέμπεται απευθείας ως `EtlResult` (direct path).

CoFlatMapProcessor (per subtask)



flatMap1 (EtlConfig):

1. configState
.put(jobId, config)
2. Build element functions
from config.transformations
→ cache in
transformationCache
3. Emit EtlResult
(config status, aggType=null)

flatMap2 (SensorEvent):

- for each config in configState:
1. Get cached element fns
from transformationCache
 2. Apply chain serially:
event → fn1 → fn2 → ...
 3. If survived & has aggs:
for each aggregation: sensor match?
YES → emit EtlResult (aggType set)
NO → skip
 4. If survived & no aggs
(or no agg matched):
emit EtlResult (aggType=null)

Διάγραμμα 2: CoFlatMap Operator

4.2.2 Εφαρμογή μετασχηματισμών

Σε αυτή την ενότητα περιγράφεται ο τρόπος με τον οποίο έχουν υλοποιηθεί οι μετασχηματισμοί. Ο χρήστης μπορεί να υποβάλει ένα ή περισσότερα αρχεία μετασχηματισμών. Κάθε αρχείο μπορεί να περιέχει έναν ή περισσότερους μετασχηματισμούς.

4.2.2.1 Μετασχηματισμοί (Transformations)

Το σύστημα υποστηρίζει δύο κατηγορίες μετασχηματισμών:

- **Element Transformations**

Τα element transformations εφαρμόζονται σειριακά ως αλυσίδα πάνω σε κάθε event. Η έξοδος κάθε transformation γίνεται η είσοδος του επόμενου. Οι transformation functions δημιουργούνται μία φορά (στο flatMap1). Η σειρά εφαρμογής καθορίζεται από τη σειρά εμφάνισης στο configuration. Αν ένα φίλτρο απορρίψει την εγγραφή η επεξεργασία σταματά για αυτό το configuration.

- **Aggregation Transformations**

Εφαρμόζονται σε ομάδες εγγραφών εντός του χρονικού παραθύρου που ορίζεται κατά την εκτέλεση της flink διαδικασίας. Τα aggregation transformations (sum, max, min, avg) εφαρμόζονται μόνο σε events που πέρασαν επιτυχώς τα element transformations. Αν ένα event κοπεί από filter, δεν φτάνει ποτέ στα aggregations.

4.2.2.2 Ποι εφαρμογής μετασχηματισμών

Η εφαρμογή μετασχηματισμών ακολουθεί διακριτά στάδια επεξεργασίας. Όταν ένα sensor event εισέρχεται στον CoFlatMap processor, πρώτα εφαρμόζονται τα element transformations που υπάρχουν αποθηκευμένα για κάθε αρχείο απαιτήσεων με την σειρά που υπάρχουν στο αρχείο. Κάθε transformation λαμβάνει ως είσοδο το αποτέλεσμα του προηγούμενου, δημιουργώντας μια αλυσίδα επεξεργασίας.

Τα element transformations χωρίζονται σε τρεις κατηγορίες: τα **filters** (filter_greater, filter_less) αποφασίζουν αν η εγγραφή θα προχωρήσει στο επόμενο στάδιο ή θα απορριφθεί, το **normalize** μετασχηματίζει αριθμητικές τιμές σε κανονικοποιημένη κλίμακα [0,1], και τα **text operations** (to_lowercase, to_uppercase, trim_whitespace) επεξεργάζονται πεδία κειμένου. Αν ένα filter απορρίψει την εγγραφή, η επεξεργασία για τη συγκεκριμένη εγγραφή εισόδου σταματά.

Μετά την ολοκλήρωση των element transformations, οι εγγραφές που προορίζονται για aggregation εισέρχονται στο στάδιο windowing που περιγράφεται εκτενώς στο επόμενο κεφάλαιο. Χρησιμοποιήθηκαν tumbling windows σταθερής διάρκειας, όπου κάθε εγγραφή ανήκει σε ακριβώς ένα παράθυρο. Εντός κάθε παραθύρου, οι εγγραφές ομαδοποιούνται με βάση το πεδίο keyBy από το αρχείο των απαιτήσεων, αν δεν έχει συμπληρωθεί από τον χρήστη τότε χρησιμοποιείται σαν default ο τύπος αισθητήρα, και υπολογίζεται η ζητούμενη συνάθροιση (sum,

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

Formatted: English (United States)

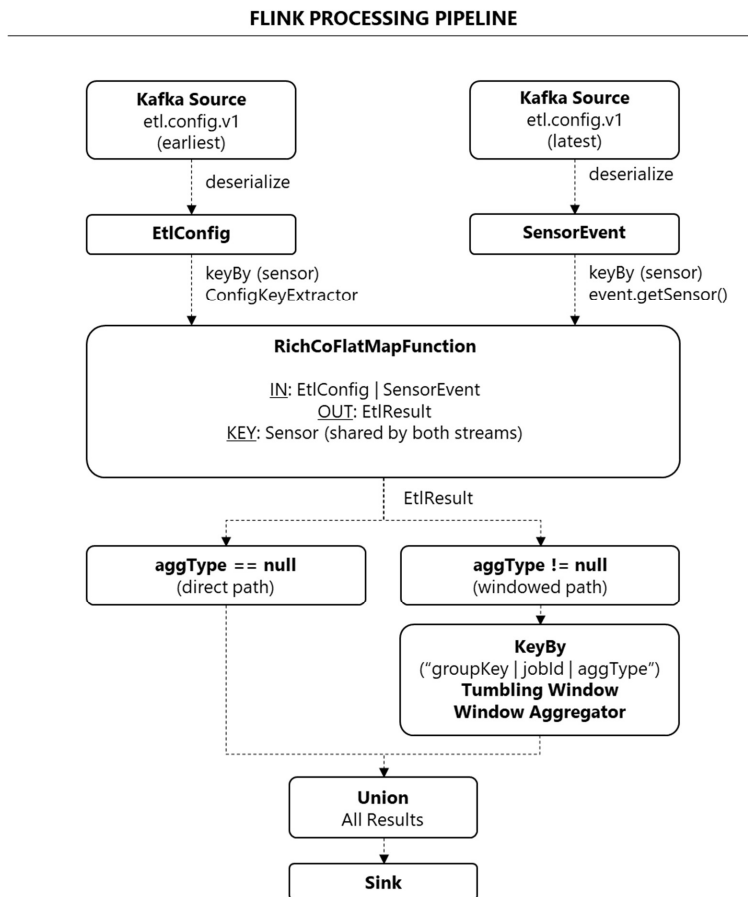
max, min, avg). Το αποτέλεσμα εκπέμπεται όταν κλείσει το παράθυρο. Αν δεν υπάρχουν μετασχηματισμοί με παράθυρο τότε τα αποτελέσματα γράφονται κατευθείαν στην έξοδο.

Τα element transformations εφαρμόζονται εντός του CoFlatMap. Τα aggregation transformations (sum, max, min, avg) απαιτούν windowing και εκτελούνται σε επόμενο στάδιο του pipeline.

Μετά τον CoFlatMap, η ροή διαχωρίζεται με βάση το πεδίο aggregationType του EtlResult:

- **Results χωρίς aggregation:** Προωθούνται απευθείας στην έξοδο, μαζί με το μήνυμα κατάστασης του config. "Configuration Submit Successfully".
- **Results με aggregation:** Περνούν από keyBy → Window → Aggregate πριν γραφτούν στην έξοδο.

Τέλος, οι δύο ροές συνενώνονται και γράφονται στην έξοδο, όπως φαίνεται και στο παρακάτω διάγραμμα.



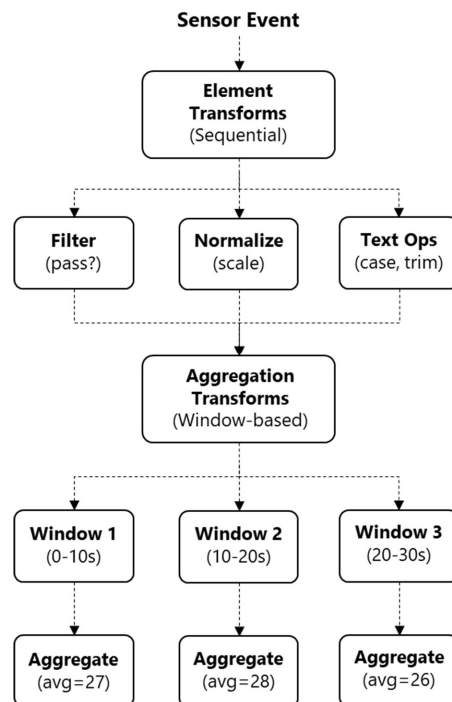
Διάγραμμα 3: Flink Processing Pipeline

4.2.3 Παράθυρα

Όπως αναφέρθηκε και παραπάνω χρησιμοποιήθηκαν tumbling windows σταθερού μεγέθους. Το μέγεθος του παραθύρου ορίζεται κατά την αρχικοποίηση του flink job. Τα aggregation results ομαδοποιούνται με ένα σύνθετο κλειδί (groupingKey|jobld|aggregationType) και εισέρχονται στο παράθυρο. Το grouping key ορίζεται σαν παράμετρος από τον χρήστη στον aggregate μετασχηματισμό. Αν δεν υπάρχει, τότε θεωρείται ως default η παράμετρος του αισθητήρα. Με τη χρήση του παραπάνω σύνθετου κλειδιού εξασφαλίζουμε ότι κάθε μοναδικός συνδυασμός ομαδοποίησης, εργασίας και τύπου συνάθροισης διατηρεί ανεξάρτητη κατάσταση (state) εντός του παραθύρου. Αυτό είναι απαραίτητο διότι ο WindowAccumulator, ο οποίος συσσωρεύει τις τιμές κατά τη διάρκεια του παραθύρου, προϋποθέτει ότι όλα τα στοιχεία που εισέρχονται σε

Commented [AD16]: Σε ολόκληρο το κεφάλαιο 3, όπου είναι οι απαιτήσεις, ούτε που το είχες αναφέρει...

αυτών ανήκουν στην ίδια ομάδα και υπολογίζονται με τον ίδιο τύπο συνάθροισης. Χωρίς το σύνθετο κλειδί, δεδομένα από διαφορετικούς αισθητήρες ή διαφορετικές εργασίες θα αναμειγνύονταν στον ίδιο accumulator, οδηγώντας σε εσφαλμένα αποτελέσματα, όπως για παράδειγμα, μια λειτουργία μέσου όρου (avg) θα μπορούσε να συγχωνευτεί με μια λειτουργία αθροίσματος (sum), ή μετρήσεις θερμοκρασίας από διαφορετικές τοποθεσίες θα συναθροίζονταν μαζί. Τέλος, ο WindowAggregator παράγει το τελικό αποτέλεσμα στο κλείσιμο κάθε παραθύρου.



Διάγραμμα 4: Μονοπάτι συναθροίσεων

4.2.4 Αποθήκευση αποτελεσμάτων

Τα τελικά αποτελέσματα που παράγονται για κάθε για κάθε ένα αρχείο απαιτήσεων αποθηκεύονται σε μια βάση δεδομένων mongo db. Κάθε αρχείο / JobId αποτελεί ένα ξεχωριστό Collection στην βάση. Παρακάτω (εικόνα 1) παρατίθεται ένα δείγμα εξόδου.

```
{
  "jobId": "avgWarmRoom-job",
  "transformations": [
    {"type": "filter_greater", "params": {"sensor": "temperature", "field": "measurement", "threshold": 25}},
    {"type": "to_uppercase", "params": {"field": "location"}},
    {"type": "avg", "keyBy": "location", "params": {"field": "measurement", "sensor": "temperature"}}
  ],
  "groupingField": "location",
  "groupingKey": "KITCHEN",
  "aggregationType": "avg",
  "field": "measurement",
  "result": 27.17,
  "processedAt": "2026-01-18T14:32:50.123Z",
  "windowStart": "2026-01-18T14:32:40.000Z",
  "windowEnd": "2026-01-18T14:32:50.000Z",
  "sensorType": "temperature",
  "measurementUnit": "Celsius",
  "location": "KITCHEN"
}
```

Εικόνα 1. Παράδειγμα εξόδου

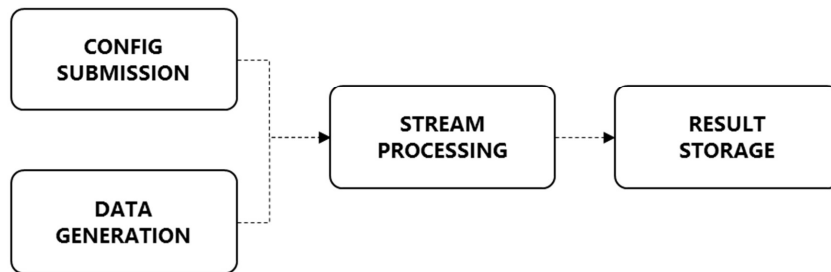
Το παραπάνω αποτέλεσμα μας δίνει την τοποθεσία σε κεφαλαία, καθώς και τον μέσο όρο θερμοκρασίας στην τοποθεσία kitchen, για θερμοκρασίες που ξεπερνούν τους 25°C, κατά το συγκεκριμένο παράθυρο χρόνου.

4.3 Ποή Δεδομένων End-to-End

Η πλήρης ροή δεδομένων από την υποβολή configuration έως την αποθήκευση αποτελεσμάτων είναι η εξής:

1. **Config Submission:** Ο χρήστης στέλνει POST στο /config με JSON configuration. Το ETL API επικυρώνει και δημοσιεύει στο config topic.
2. **Data Generation:** Ο Producer δημιουργεί sensor events και τα δημοσιεύει στο input topic με configurable rate.
3. **Stream Processing:** Το Flink Job καταναλώνει και τα δύο streams. Ο CoFlatMap αποθηκεύει τα configs στο state και επεξεργάζεται τα events. Τα aggregations περνούν από windowing.
4. **Result Storage:** Τα αποτελέσματα αποθηκεύονται στη βάση δεδομένων MongoDB.

END – TO – END DATA FLOW

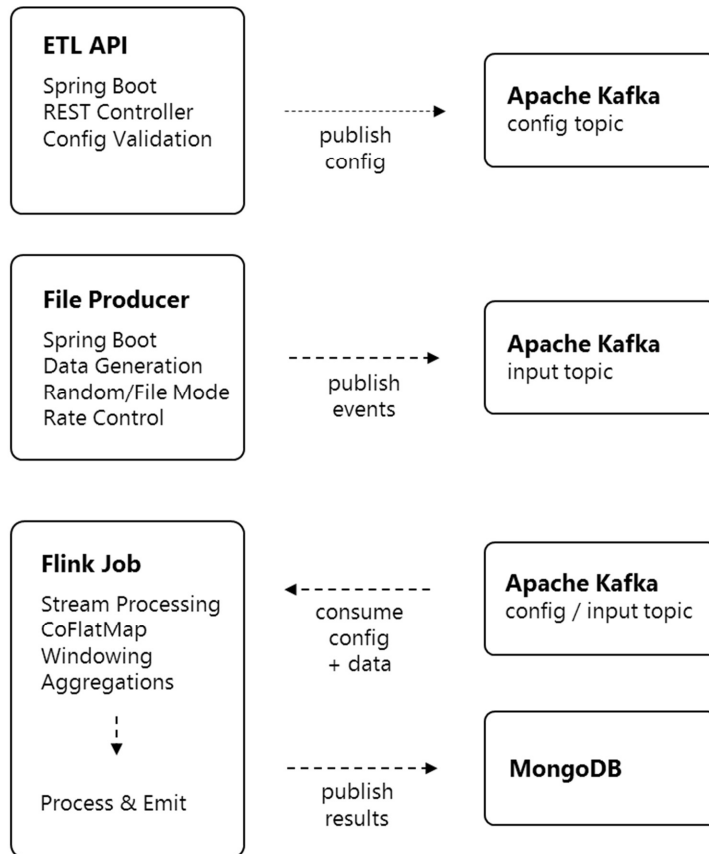


Διάγραμμα 5: End-to-end ροή δεδομένων

4.4 Βοηθητικές υπηρεσίες - Microservices

Ολόκληρη η εφαρμογή έχει στηθεί στην λογική των microservices. Εκτός από το Flink Job που αναλύθηκε παραπάνω, αναπτύχθηκαν ακόμα δύο βοηθητικές υπηρεσίες για την εξυπηρέτηση του σκοπού της εργασίας. Η υπηρεσία file producer που δημιουργεί τα δεδομένα των αισθητήρων, και η υπηρεσία ETL API, που επιτρέπει στον χρήστη να υποβάλλει τα ερωτήματά του προς την ροή των δεδομένων. Κάθε microservice εκτελείται σε ξεχωριστό Docker container και επικοινωνεί με τα υπόλοιπα μέσω Apache Kafka, όπως φαίνεται και στο διάγραμμα 6.

MICROSERVICES ARCHITECTURE



Διάγραμμα 6: Αρχιτεκτονική microservices

4.4.1 ETL API Service

Το ETL API service είναι ένα REST API υλοποιημένο με το framework Spring Boot. Αποτελεί το σημείο εισόδου του συστήματος για τους χρήστες. Δέχεται configurations σε JSON μορφή μέσω HTTP POST στο endpoint /config. Κάθε configuration περιέχει ένα μοναδικό jobId και μια λίστα μετασχηματισμών προς εφαρμογή.

Η υπηρεσία εκτελεί επικύρωση του configuration: ελέγχει ότι υπάρχει το jobId, ότι οι τύποι μετασχηματισμών είναι έγκυροι, και ότι οι απαραίτητες παράμετροι υπάρχουν για κάθε τύπο. Μετά την επικύρωση, το configuration δημοσιεύεται στο Kafka στο config topic.

Endpoint	Method	Περιγραφή
/config	POST	Υποβολή configuration - επικύρωση και δημοσίευση στο Kafka
/health	GET	Health check για container orchestration

Πίνακας 5: REST endpoints του ETL API

Παρακάτω ακολουθεί ένα δείγμα Configuration File που κατατίθεται (Post) από την υπηρεσία. Το συγκεκριμένο παράδειγμα configuration file φιλτράρει τις θερμοκρασίες μεγαλύτερες των 25°C, υπολογίζει τον μέσο όρο θερμοκρασίας ανά τοποθεσία, και μετατρέπει το πεδίο της τοποθεσίας σε κεφαλαία γράμματα.

```
{
  "jobId": "avgWarmRoom-job",
  "transformations": [
    {
      "type": "filter_greater",
      "params": {
        "sensor": "temperature",
        "field": "measurement",
        "threshold": 25
      }
    },
    {
      "type": "to_uppercase",
      "params": {
        "field": "location"
      }
    },
    {
      "type": "avg",
      "keyBy": "location",
      "params": {
        "field": "measurement",
        "sensor": "temperature"
      }
    }
  ]
}
```

Εικόνα 2. Παράδειγμα πλήρους Configuration.

4.4.2 File Producer Service

Η υπηρεσία File Producer παράγει δεδομένα αισθητήρων και τα δημοσιεύει στο input topic του Kafka. Πρόκειται ουσιαστικά για την ροή εισόδου του συστήματός μας. Στόχος της είναι να προσομοιώσει ένα ρεαλιστικό δίκτυο αισθητήρων με 6 τύπους μετρήσεων (θερμοκρασία, πίεση, υγρασία, ποιότητα αέρα, φωτισμός και θόρυβος) σε 8 διαφορετικές τοποθεσίες.

Σε κάθε κύκλο παραγωγής, δημιουργούνται μετρήσεις για όλους τους συνδυασμούς αισθητήρων και τοποθεσιών, δηλαδή 48 μηνύματα (8 τοποθεσίες × 6 αισθητήρες). Οι τιμές των μετρήσεων

ακολουθούν κανονική κατανομή (Gaussian) με ρεαλιστικά όρια για κάθε τύπο αισθητήρα. Κάθε μήνυμα περιλαμβάνει τον τύπο αισθητήρα, την τιμή μέτρησης, τη μονάδα μέτρησης, την τοποθεσία, το timestamp και έναν δείκτη ποιότητας δεδομένων που υπολογίζεται με βάση την απόκλιση από τις αναμενόμενες τιμές. Ο ρυθμός παραγωγής είναι παραμετροποιήσιμος μέσω environment variable (π.χ. RATE_PER_SEC=10 για 10 κύκλους ανά δευτερόλεπτο, δηλαδή 480 μηνύματα/sec).

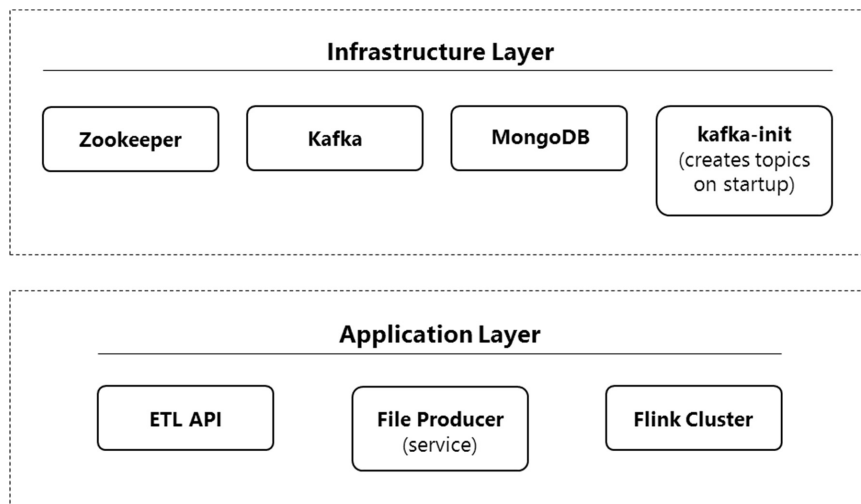
```
{
  "sensor": "temperature",
  "measurement": 22.1,
  "measurement_unit": "Celsius",
  "datetime": "2026-01-18T14:32:45Z",
  "location": "kitchen",
  "data_quality": "excellent"
}
```

Εικόνα 3. Δεδομένα εισόδου

4.5 Docker Containerization

Η υποδομή βασίζεται σε Docker containers. Το Docker Compose ορίζει τις υπηρεσίες και τις εξαρτήσεις τους. Η στοίβα οργανώνεται σε δύο επίπεδα:

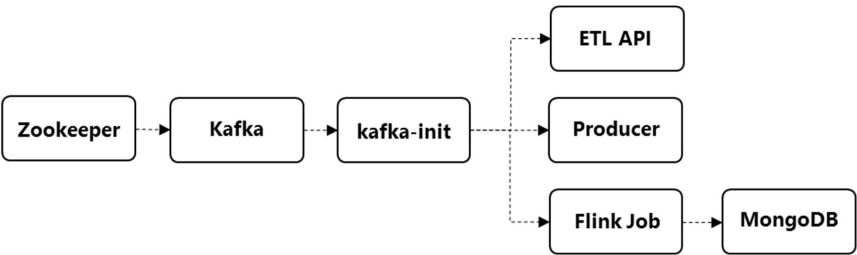
- **Infrastructure Layer:** Zookeeper, Kafka, MongoDB - οι υπηρεσίες υποστήριξης
- **Application Layer:** ETL API, File Producer, Flink - τα microservices της εφαρμογής



Διάγραμμα 7: Docker Compose stack

Commented [AD17]: Γενικώς, δεν αναφέρεσαι συχνά στο κείμενο στις εικόνες, στα διαγράμματα κτλ.

Η σειρά εκκίνησης καθορίζεται από τις παρακάτω εξαρτήσεις:



Διάγραμμα 8: Εξαρτήσεις υπηρεσιών

4.6 Τεχνολογικό Stack

Οι τεχνολογίες που χρησιμοποιήθηκαν φαίνονται στον παρακάτω πίνακα.

Επίπεδο	Τεχνολογία	Ρόλος
Runtime	Java	Γλώσσα προγραμματισμού
Framework	Spring Boot	REST API και Producer services
Stream Processing	Apache Flink	Stateful stream processing
Message Broker	Apache Kafka	Event streaming μεταξύ services
Coordination	Apache Zookeeper	Kafka cluster coordination
Database	MongoDB	Αποθήκευση αποτελεσμάτων
Containerization	Docker	Container runtime
Orchestration	Docker Compose	Multi-container deployment

Πίνακας 6: Τεχνολογικό stack

Commented [AD18]: Υπερβολικά σύντομος...

Κεφάλαιο 5

Εξαγωγή πειραμάτων

Η πειραματική αξιολόγηση του συστήματος ETL πραγματοποιήθηκε στο κατανεμημένο περιβάλλον του SoftNet Cluster του Πολυτεχνείου Κρήτης. Ο cluster αποτελείται από 11 όμοια μηχανήματα Dell PowerEdge R300, τα οποία διαθέτουν τα παρακάτω χαρακτηριστικά:

- Quad Core Xeon X3323 2.5GHz, 11 μηχανήματα με 3 task slots το καθένα
- 8GB RAM
- 500GB HDD

5.1 Πειραματικά αποτελέσματα

Για την εκτέλεση των πειραμάτων, ακολουθήθηκε η μέθοδος της προδημοσίευσης. Εισήχθησαν τα δεδομένα εισόδου και οι ενεργές ρυθμίσεις στα Kafka topics, και στην συνέχεια εκτελέστηκε το Flink Job. Οι μετρήσεις έγιναν παρακολουθώντας τα offsets του Kafka consumer group του Flink.

- Ο χρόνος εκκίνησης καταγράφηκε όταν ο consumer άρχισε να καταναλώνει εγγραφές.
- Ο χρόνος τερματισμού καταγράφηκε όταν το output topic σταμάτησε να δέχεται νέες εγγραφές.

Χρειάστηκε να τροποποιηθούν τα ακόλουθα:

- Το Sink εξόδου του Flink Job, το οποίο τροποποιήθηκε από MongoDB Sink σε Kafka Sink.
- Η υπηρεσία file producer, έτσι ώστε να δημοσιεύει N αριθμό εγγραφών στο output topic και να τερματίζει.

Αρχικά, λοιπόν, θεωρήθηκαν σαν σταθερές τιμές επαρκούς απόδοσης οι ακόλουθες

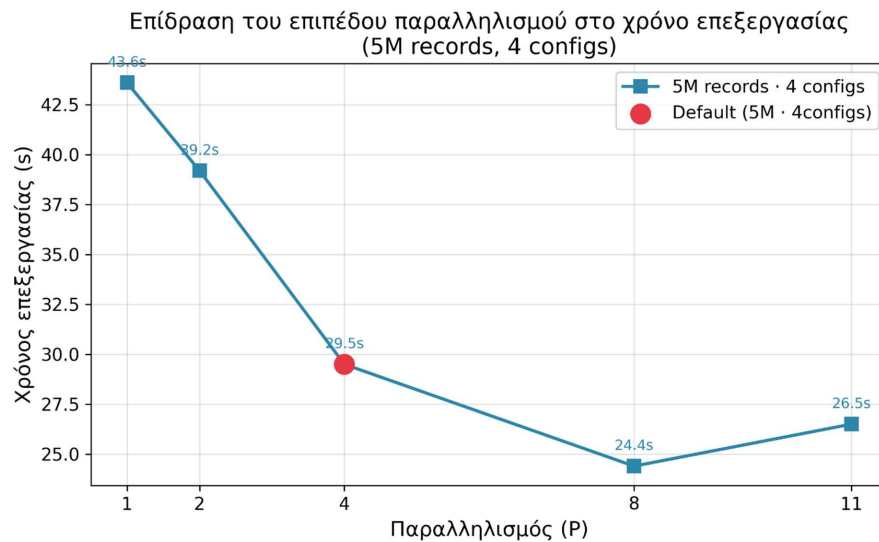
- Τα πέντε εκατομμύρια εγγραφών εισόδου,
- Τα τέσσερα ενεργά configurations,
- Επίπεδο παραλληλισμού στο flink ίσο με τέσσερα.

Για να αξιολογηθεί η απόδοση του job μας, εκτελέστηκαν τέσσερις διαφορετικές κατηγορίες πειραμάτων. Πρώτα μελετήθηκε τι γίνεται αν μεταβληθεί το επίπεδο παραλληλισμού, στη συνέχεια τι γίνεται αν μεταβληθεί το πλήθος των δεδομένων εισόδου, και τέλος τι γίνεται αν μεταβληθούν τα ενεργά configurations, καθώς και τι συμβαίνει αν αυξηθεί η πολυπλοκότητα των ρυθμίσεων αυτών. Παρακάτω ακολουθούν αναλυτικά τα αποτελέσματα.

5.1.1 Επίδραση του επιπέδου παραλληλισμού

Σε αυτή την κατηγορία πειραμάτων μεταβλήθηκε το επίπεδο παραλληλισμού, ενώ ο όγκος δεδομένων (5M εγγραφές) και ο αριθμός ρυθμίσεων (4 configurations) παρέμειναν σταθεροί. Σε κάθε τρέξιμο ο αριθμός Kafka partitions ισούται με τον βαθμό παραλληλισμού, ώστε κάθε subtask να εξυπηρετεί αποκλειστικά ένα partition. Τα αποτελέσματα, όπως παρουσιάζονται στο Διάγραμμα Z, καταδεικνύουν sub-linear κλιμάκωση: η μετάβαση από P=1 (43.6s) σε P=4 (29.5s) επιφέρει επιτάχυνση μόλις ~1.5x, αντί του θεωρητικού 4x. Περαιτέρω αύξηση σε P=8 (24.4s) φέρνει μικρή επιπλέον βελτίωση, ενώ στο P=11 ο χρόνος ανεβαίνει ελαφρά σε 26.5s.

Η υπο-γραμμική αυτή συμπεριφορά ερμηνεύεται από δύο παράγοντες. Πρώτον, το aggregate throughput του απομακρυσμένου Kafka cluster αποτελεί σταθερό ανώτατο όριο στον ρυθμό εισαγωγής δεδομένων: ακόμα και όταν αυξάνεται το P και περισσότερα subtasks διαβάζουν παράλληλα, το network bandwidth μοιράζεται ανάμεσα σε όλους, εμποδίζοντας ιδανική κλιμάκωση. Δεύτερον, ο producer παράγει δεδομένα για 6 μόνο sensor types, οπότε το keyBy(sensor) κατανέμει events σε το πολύ 6 διακριτά CoFlatMap subtasks. Από P=8 και πάνω, τουλάχιστον 2 subtasks παραμένουν idle, αποτελώντας ανώτατο όριο ωφέλιμης κλιμάκωσης — ανεξαρτήτως διαθέσιμων πόρων.

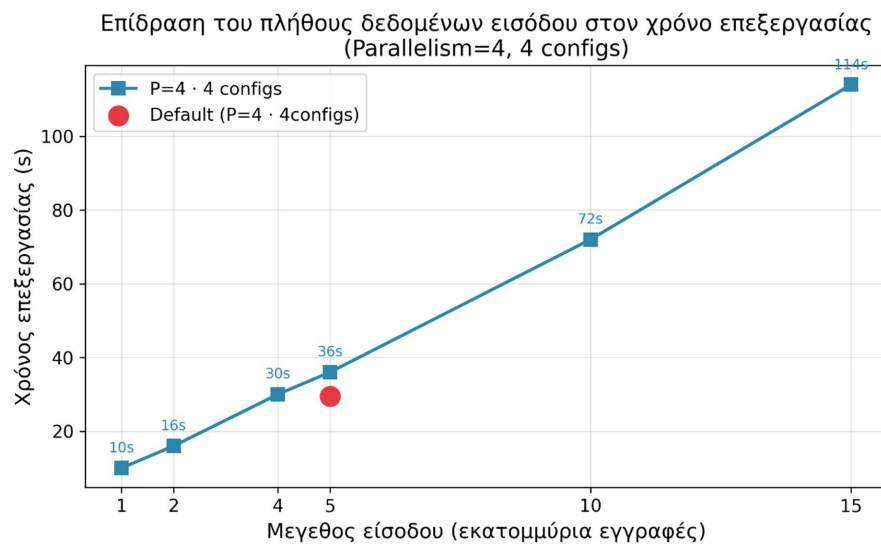


Διάγραμμα 9: Επίδραση επιπέδου παραλληλισμού

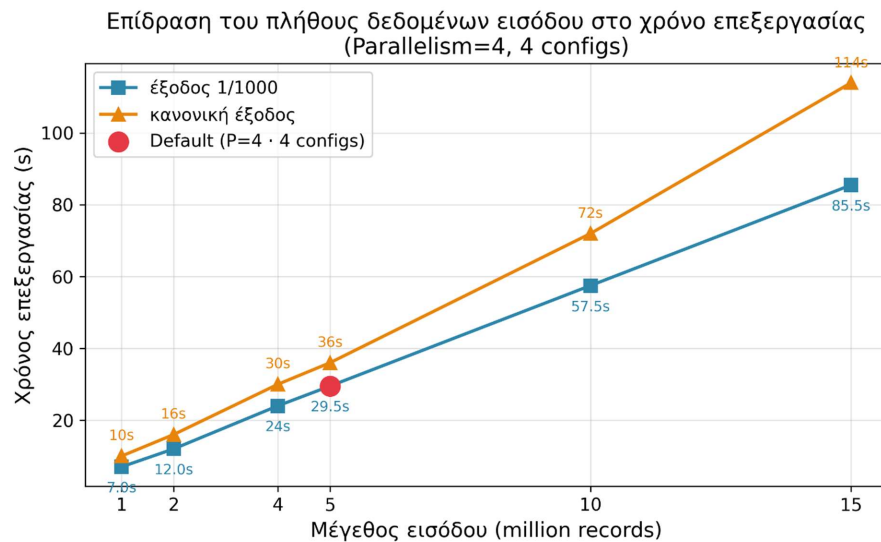
5.1.2 Επίδραση του μεγέθους δεδομένων εισόδου

Σε αυτή την κατηγορία πειραμάτων μεταβλήθηκε το πλήθος των δεδομένων εισόδου, ενώ ο βαθμός παραλληλισμού ($P=4$) και ο αριθμός των ενεργών ρυθμίσεων (4 configurations) παρέμειναν σταθεροί. Τα αποτελέσματα, όπως παρουσιάζονται στο Διάγραμμα 10, δείχνουν ότι ο χρόνος επεξεργασίας κλιμακώνεται γραμμικά με τον όγκο εισόδου: από 10s για 1M εγγραφές, σε 36s για 5M (σημείο αναφοράς), έως 114s για 15M εγγραφές, με σταθερή κλίση περίπου 7s ανά εκατομμύριο εγγραφές. Η γραμμική αυτή συμπεριφορά επιβεβαιώνει ότι το κόστος επεξεργασίας ανά εγγραφή παραμένει σταθερό ανεξαρτήτως όγκου.

Στο ίδιο πλαίσιο εξετάστηκε η επίδραση του Kafka sink μέσω σύγκρισης δύο σεναρίων: κανονικής εξόδου, όπου τα αποτελέσματα αποστέλλονται πλήρως στο Kafka output topic, και δειγματοληψίας εξόδου (1/1000), όπου μόνο ένα κλάσμα των αποτελεσμάτων γράφεται στο sink. Όπως παρουσιάζεται στο Διάγραμμα 11, η διαφορά είναι σταθερά ορατή σε όλο το εύρος δεδομένων: στο 1M η δειγματοληψία μειώνει τον χρόνο από 10s σε 7.8s, στο 5M από 36s σε 29.5s, και στο 15M από 114s σε 85.5s. Το γεγονός ότι η απόκλιση μεγαλώνει με τον όγκο υποδηλώνει ότι το κόστος εγγραφής στο Kafka είναι αναλογικό προς τον αριθμό εγγραφών εξόδου, επιβεβαιώνοντας ότι το network I/O προς το Kafka cluster αποτελεί το κυρίαρχο bottleneck του συστήματος, και όχι ο υπολογιστικός πυρήνας.



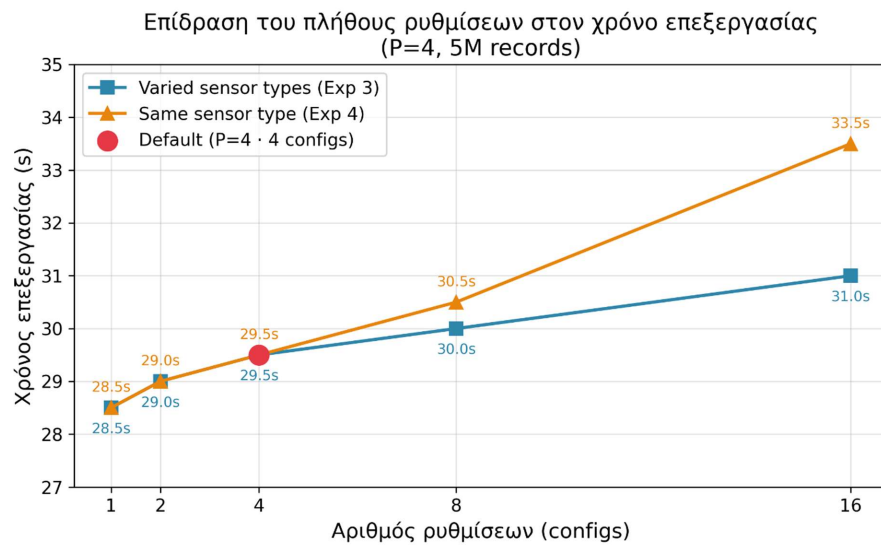
Διάγραμμα 10. Επίδραση δεδομένων εισόδου



Διάγραμμα 11. Επίδραση δεδομένων εισόδου (1/1000 sink)

5.1.3 Επίδραση του αριθμού ενεργών configurations

Σε αυτή την κατηγορία πειραμάτων μεταβλήθηκε ο αριθμός των ενεργών ρυθμίσεων, εξετάζοντας δύο σενάρια: ρυθμίσεις για διαφορετικούς αισθητήρες (διάγραμμα 11-μπλε γραμμή) και ρυθμίσεις για τον ίδιο αισθητήρα (διάγραμμα 11-πορτοκαλί γραμμή), ενώ ο βαθμός παραλληλισμού ($P=4$) και ο όγκος δεδομένων (5M εγγραφές) παρέμειναν σταθεροί. Τα αποτελέσματα, όπως παρουσιάζονται στο Διάγραμμα W, δείχνουν ότι στο σενάριο με διαφορετικούς αισθητήρες ο αριθμός ρυθμίσεων επηρεάζει ελάχιστα τον χρόνο εκτέλεσης: από 28.5s για 1 ρύθμιση σε 31.0s για 16 ρυθμίσεις. Αντίθετα, στο σενάριο με τον ίδιο αισθητήρα η επίδραση γίνεται εμφανής για μεγάλο αριθμό ρυθμίσεων: 28.5s για 1 ρύθμιση έναντι 33.5s για 16 ρυθμίσεις. Τα δύο σενάρια είναι πρακτικά ταυτόσημα έως 4 ρυθμίσεις, και αποκλίνουν από τις 8 ρυθμίσεις και πάνω. Η διαφορά ερμηνεύεται από τον τρόπο κατανομής ρυθμίσεων μεταξύ subtasks. Στο πρώτο σενάριο, κάθε ρύθμιση δρομολογείται σε διαφορετικό CoFlatMap subtask βάσει του sensor key, οπότε κάθε subtask εκτελεί $O(1)$ επαναλήψεις ανά event ανεξαρτήτως αριθμού ρυθμίσεων. Στο δεύτερο σενάριο, όλες οι ρυθμίσεις φέρουν τον ίδιο sensor key και συσσωρεύονται στο ίδιο subtask, αυξάνοντας το βρόχο επεξεργασίας σε $O(N)$ ανά event. Ωστόσο, ακόμα και στη χειρότερη περίπτωση (16 ρυθμίσεις, ίδιος αισθητήρας, 33.5s), η επίπτωση παραμένει ήπια, καθώς το Kafka network I/O εξακολουθεί να αποτελεί το κυρίαρχο bottleneck.



Διάγραμμα 12: Επίδραση ενεργών configurations

Κεφάλαιο 6

Επίλογος

6.1 Συμπεράσματα

Στην παρούσα διπλωματική εργασία δημιουργήθηκε ένα ολοκληρωμένο σύστημα ETL για την επεξεργασία συνεχών ροών δεδομένων που προέρχονται από ένα δίκτυο αισθητήρων. Το σύστημα μας, επιτρέπει στους χρήστες την δυναμική υποβολή ρυθμίσεων/configurations για την επεξεργασία της συγκεκριμένης ροής.

Για την ανάγκη της εργασίας αναπτύχθηκαν οι επιμέρους υπηρεσίες που επιτρέπουν την υποβολή των ρυθμίσεων από τον χρήστη αλλά και την δημιουργία της ροής των δεδομένων. Οι παραπάνω υπηρεσίες όπως και το flink job αναπτύχθηκαν σε Docker containers.

Οι εισερχόμενες ροές δεδομένων συνδυάζονται με τη χρήση του τελεστή CoFlatMap, και πιο συγκεκριμένα του RichCoFlatMapFunction, ο οποίος παρέχει πρόσβαση σε καταμεμημένη κατάσταση μέσω του MapState. Με τον τρόπο αυτό εξασφαλίζεται ότι τόσο τα configurations όσο και τα δεδομένα εισόδου αντιστοιχίζονται στο ίδιο κλειδί (sensor) και επεξεργάζονται από το ίδιο παράλληλο subtask, καθώς και οι δύο ροές χρησιμοποιούν το ίδιο κλειδί κατανομής.

Συνολικά, τα αποτελέσματα των πειραμάτων αναδεικνύουν τα χαρακτηριστικά και τα όρια κλιμάκωσης του συστήματος. Ως προς τον όγκο δεδομένων, ο χρόνος επεξεργασίας κλιμακώνεται γραμμικά, γεγονός που επιβεβαιώνει σταθερό υπολογιστικό κόστος ανά εγγραφή, ανεξαρτήτως του συνολικού μεγέθους του dataset. Το κυρίαρχο bottleneck σε όλες τις συνθήκες εντοπίζεται στο network I/O προς το Kafka cluster, γεγονός που επιβεβαιώνεται και από τη σύγκριση κανονικής εξόδου με δειγματοληψία 1/1000. Ως προς τον παραλληλισμό, η κλιμάκωση είναι sub-linear: η μετάβαση από P=1 σε P=4 επιφέρει επιτάχυνση ~1.5x αντί του θεωρητικού 4x, με το Kafka throughput barrier και το ανώτατο όριο των 6 διακριτών sensor types να αποτελούν τους κύριους περιορισμούς. Ως προς τον αριθμό ενεργών configurations, η επίδραση εξαρτάται από το σενάριο κατανομής: για ρυθμίσεις διαφορετικών αισθητήρων η επιβάρυνση είναι αμελητέα, ενώ για ρυθμίσεις του ίδιου αισθητήρα παρατηρείται ήπια αύξηση για μεγάλο αριθμό ρυθμίσεων, χωρίς ωστόσο να αποτελεί καθοριστικό παράγοντα. Συνεπώς, η υλοποίηση διατηρεί αποδοτική και προβλέψιμη συμπεριφορά ως προς τον όγκο δεδομένων και τον αριθμό ρυθμίσεων, ενώ η βελτίωση μέσω παραλληλισμού εξαντλείται γρήγορα λόγω των δομικών χαρακτηριστικών του workload.

6.2 Μελλοντικές επεκτάσεις

Το σύστημα που αναπτύσσεται στα πλαίσια αυτής της διπλωματικής εργασίας θα μπορούσε να βελτιωθεί και να επεκταθεί περαιτέρω. Συγκεκριμένα, μπορούν να αναπτυχθούν επιπλέον συναρτήσεις μετασχηματισμών ώστε να μπορούν να εκτελεστούν περισσότερα και πιο σύνθετα

ερώτημα στην εισερχόμενη ροή δεδομένων. Επίσης, θα μπορούσε μελλοντικά να αναπτυχθεί μια διεπαφή χρήστη (UI) ώστε η υποβολή και η απεικόνιση των αποτελεσμάτων να γίνεται με πιο συμβατό τρόπο, καθώς και να προστεθούν επιπλέον rest μέθοδοι για παύση ή και τροποποίηση των ενεργών configuration files.

Βιβλιογραφία

- [1] <https://kafka.apache.org/41/getting-started/introduction/> Accessed: 2025-10-14
- [2] [https://en.wikipedia.org/wiki/Extract,_transform,_load#Extract,_load,_transform_\(ELT\)](https://en.wikipedia.org/wiki/Extract,_transform,_load#Extract,_load,_transform_(ELT))
Accessed: 2025-12-10
- [3] <https://airbyte.com/data-engineering-resources/streaming-etl> Accessed: 2025-12-13
- [4] <https://kafka.apache.org/> Accessed: 2025-10-14
- [5] <https://kafka.apache.org/41/apis/> Accessed: 2025-10-14
- [6] <https://flink.apache.org/what-is-flink/flink-architecture/> Accessed: 2025-12-12
- [7] <https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/operators/overview/#connect>
- [8] <https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/fault-tolerance/state/>